Rasterization

Introduction

- Process of turning triangles into pixels

- Layering triangles together on the screen

Forming Triangles

- Vertex shader transforms vertices and stores data in a new buffer

- Index buffer determines how they form triangles

Forming Triangles

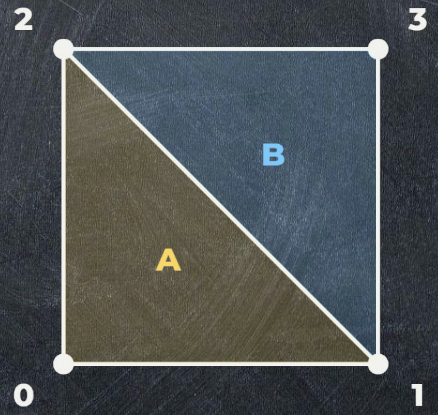
- **Entries are indices into Vertex buffer. Triplets create triangles**

Vertex Buffer

#	Position (x, y)
0	(0, 0)
1	(1, 0)
2	(0, 1)
3	(1, 1)

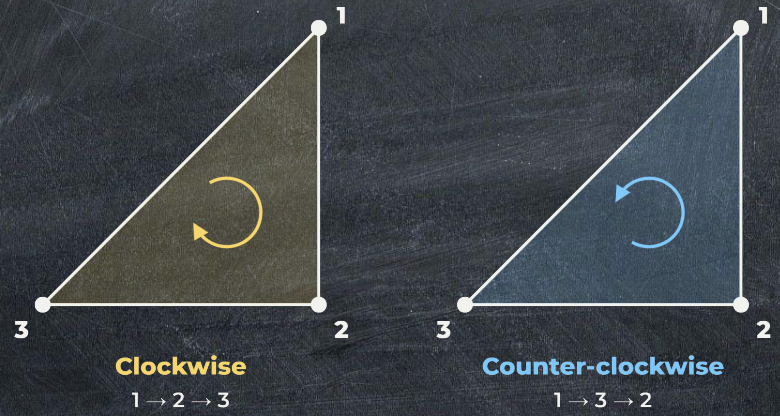
Index Buffer





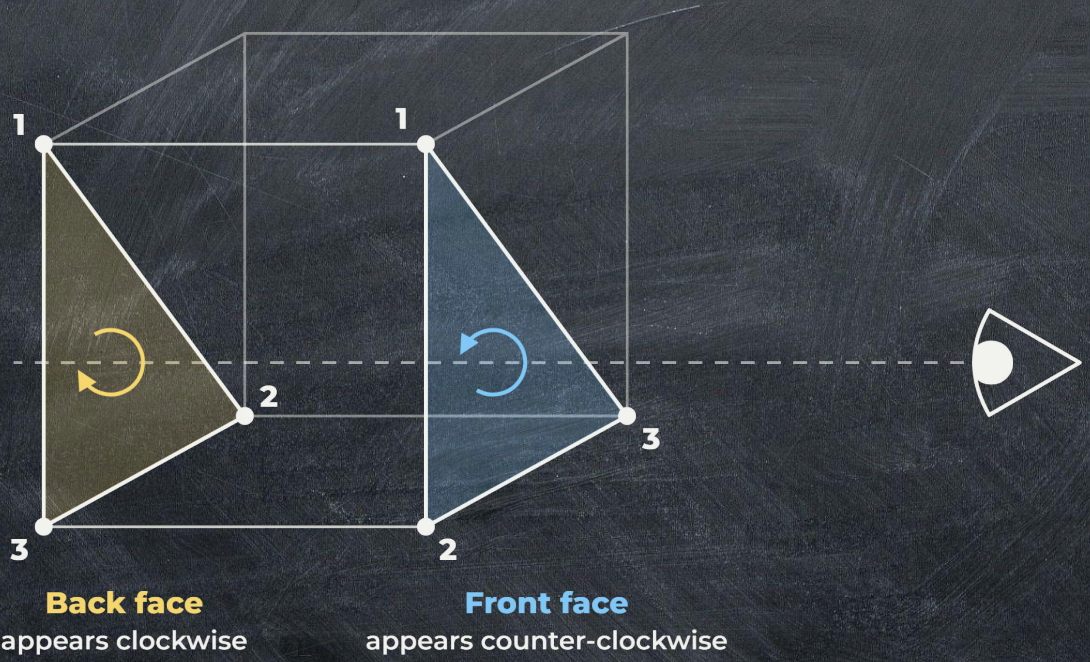
Backface Culling

- **Vertex order, “winding,” is important. Creates a front and back side.**



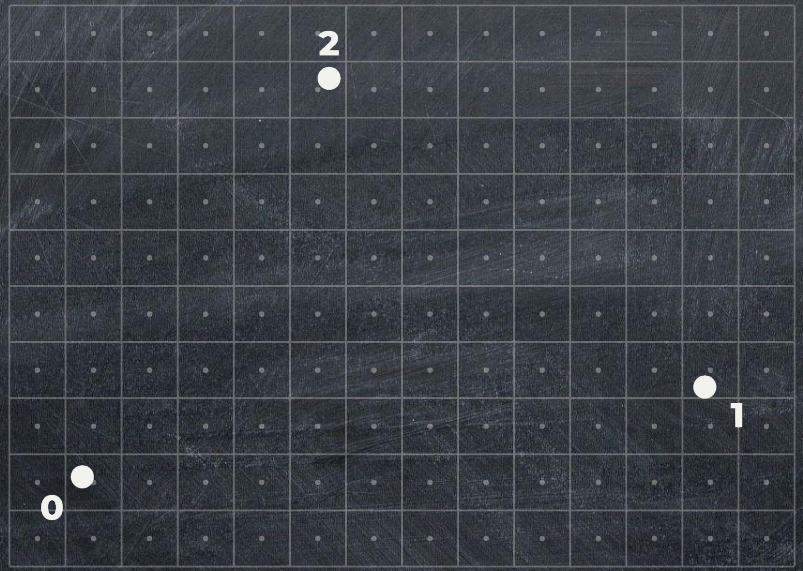
Backface Culling

- **One side is usually hidden; don’t need to render the inside of an object.**



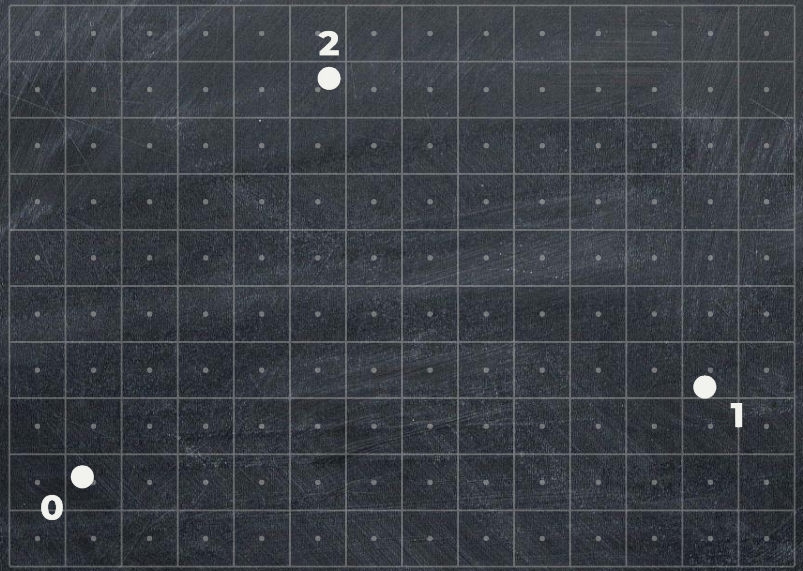
Rasterizing

- **This algorithm determines which pixels a triangle covers.**



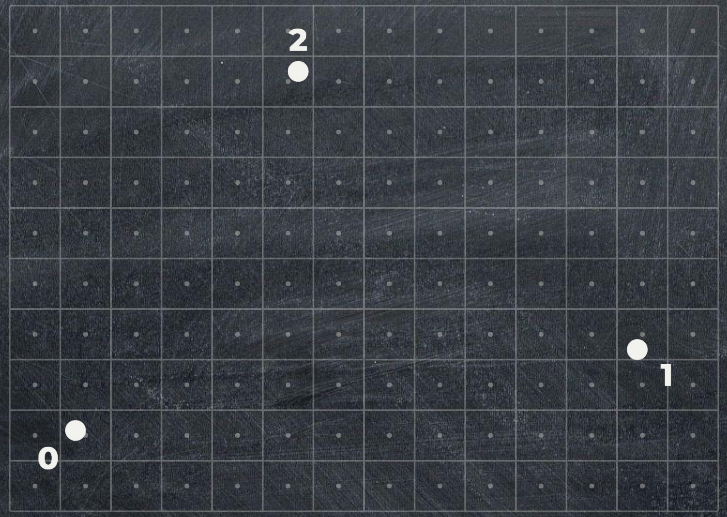
Rasterizing

- **A: place each triangle vertex on the screen and create a bounding rectangle**

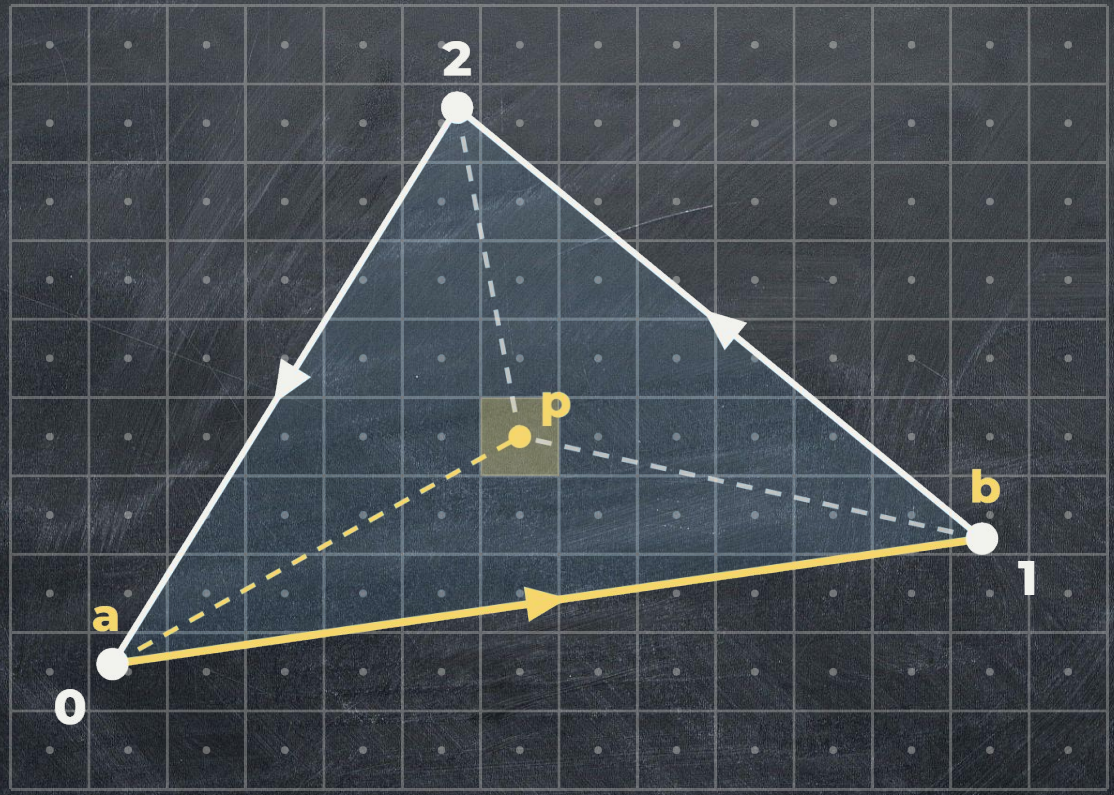


Rasterizing

- **B. Pick a pixel inside the rectangle. Determine which side of each triangle edge its center lies on.**

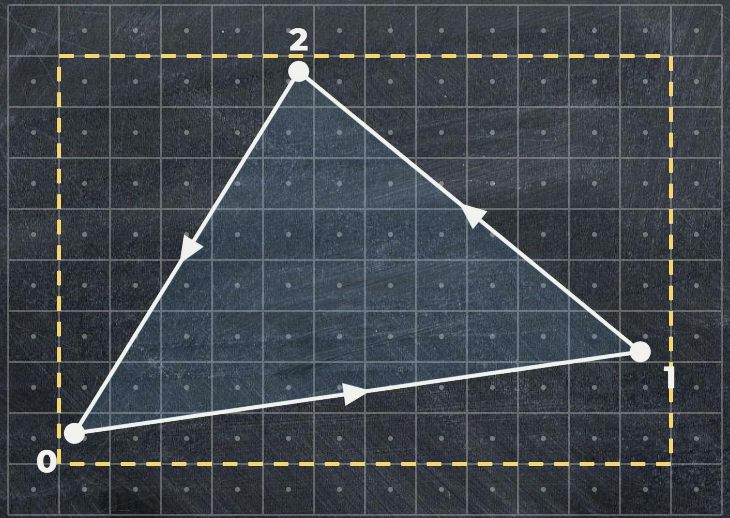


side = (b.x – a.x)(p.y – a.y) – (b.y – a.y)(p.x – a.x)

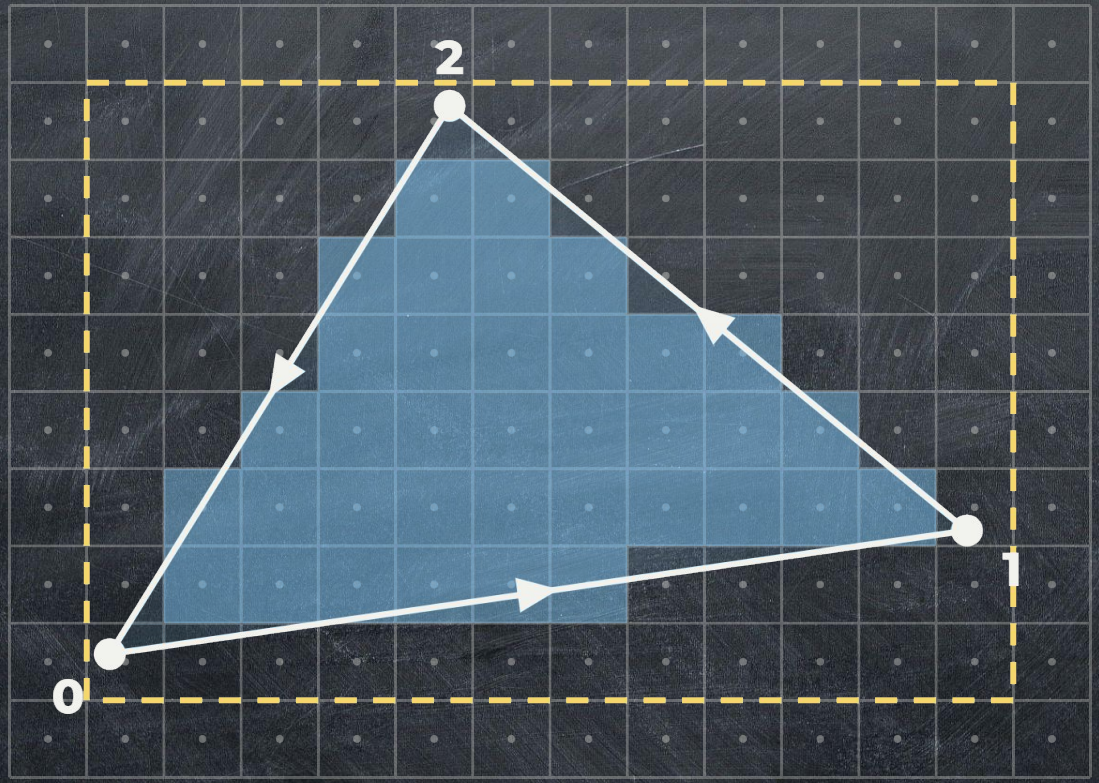


Rasterizing

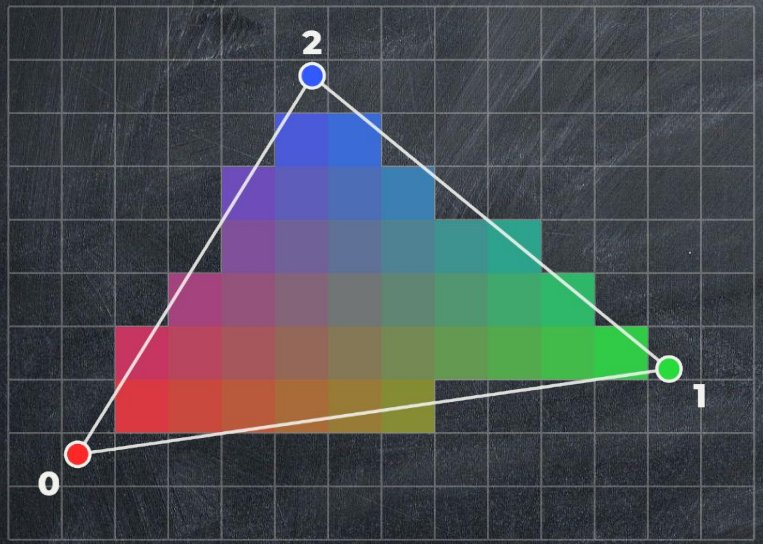
- **C. If the pixel is on the same side of each triangle edge, it is inside the triangle. Repeat for all pixels in the bounding rectangle.**



Rasterizing

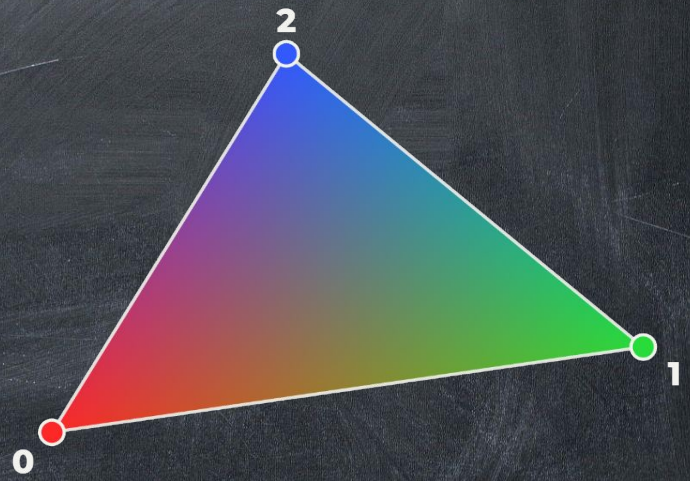


Interpolation



36 pixels

one color per pixel center

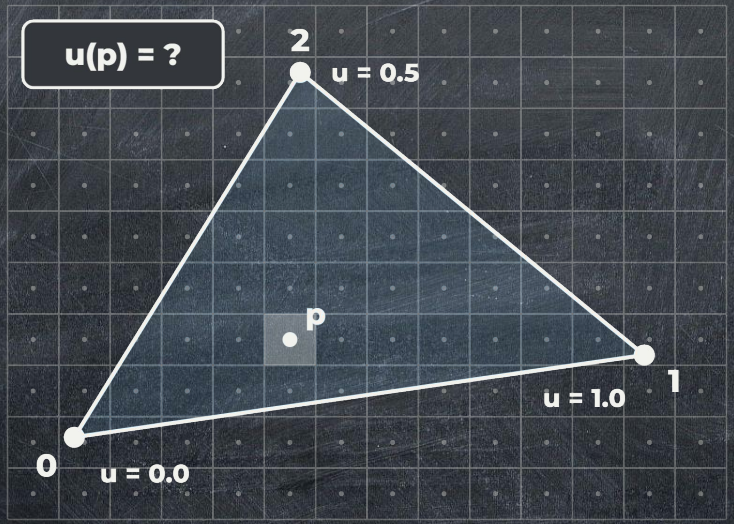


Full resolution

same math, more pixels

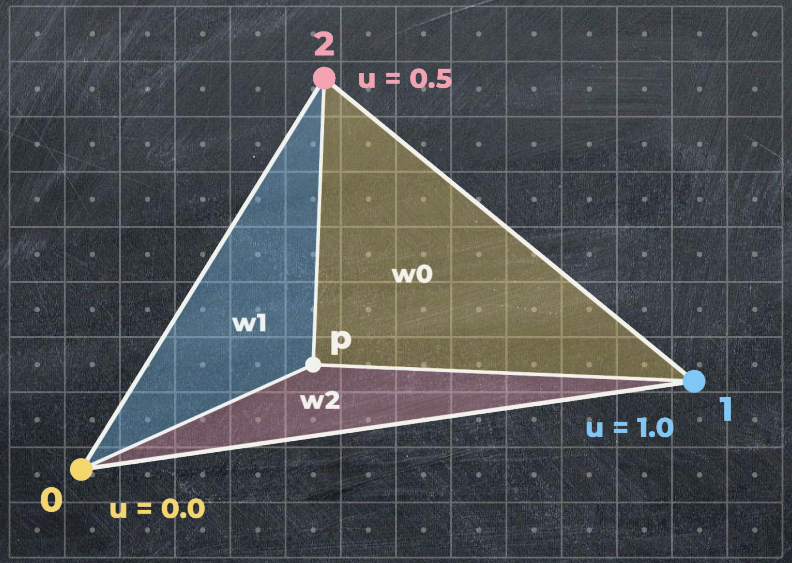
Interpolation

- We need to interpolate vertex values.
- For illustration, we'll use a tex coord: u.



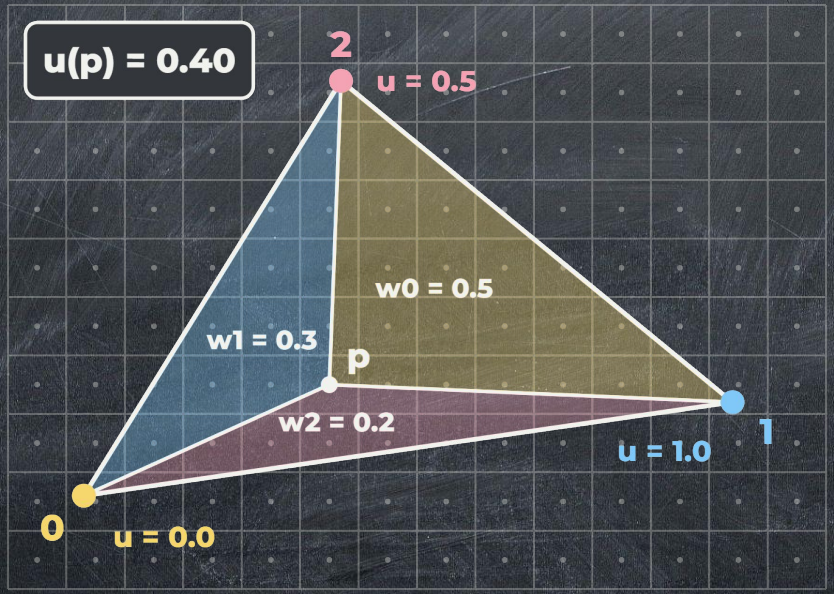
- **Each pixel gets a weighted blend of the corner verts' values. Weights sum to 1. Barycentric coordinates!**

$$u = w0 \cdot u0 + w1 \cdot u1 + w2 \cdot u2$$



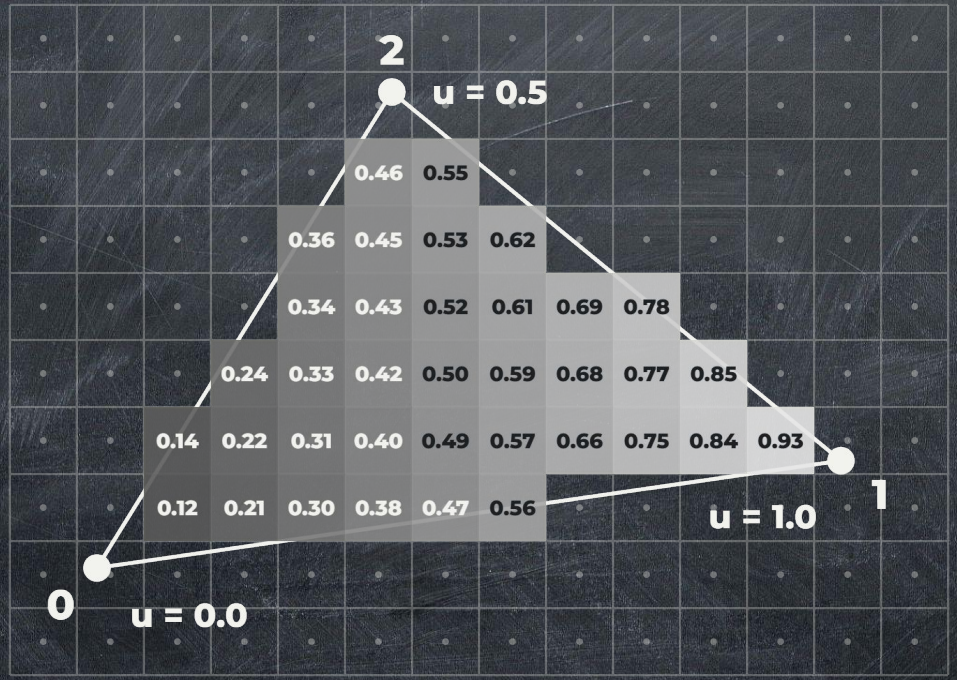
- **We already computed these weights while rasterizing!**

w0 = side12 / (side01 + side12 + side20)



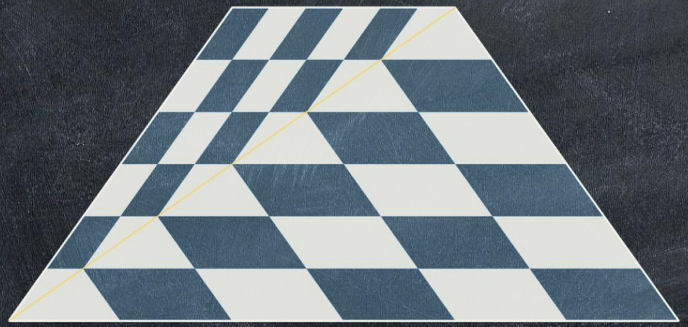
u(p) = 0.5·0.0 + 0.3·1.0 + 0.2·0.5 = 0.40

Interpolation

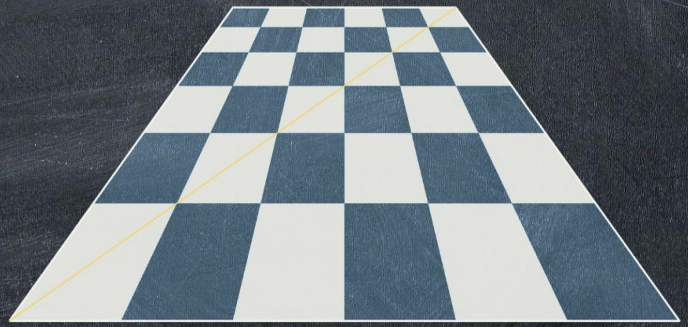


Interpolation

- **One more step to correct for perspective: divide each weight by its vertex's depth from the camera, then rescale so they add up to 1.**



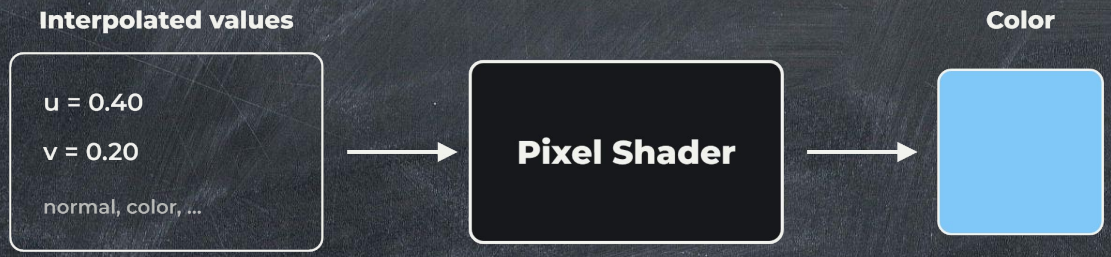
Linear
warps along the diagonal



Perspective-correct
squares shrink with distance

Pixel Shaders

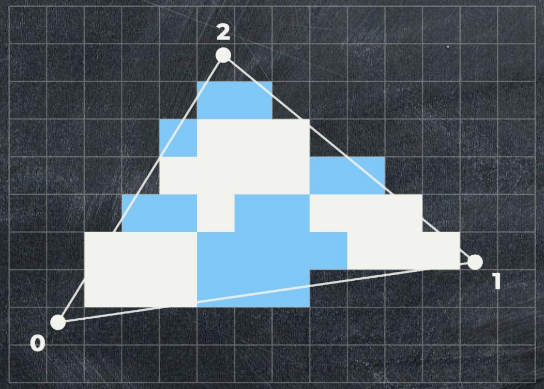
- **These take interpolated values and output colors.**



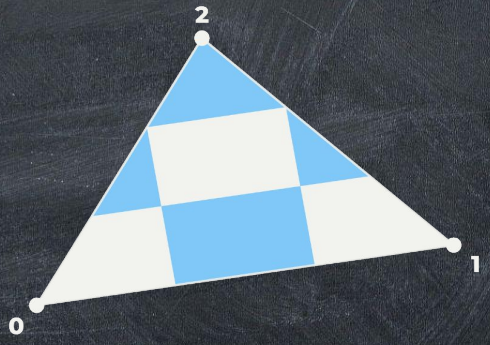
- **Runs once per pixel in the triangle.**

Pixel Shaders

- **Colors are written to the framebuffer, which eventually makes its way to the screen.**



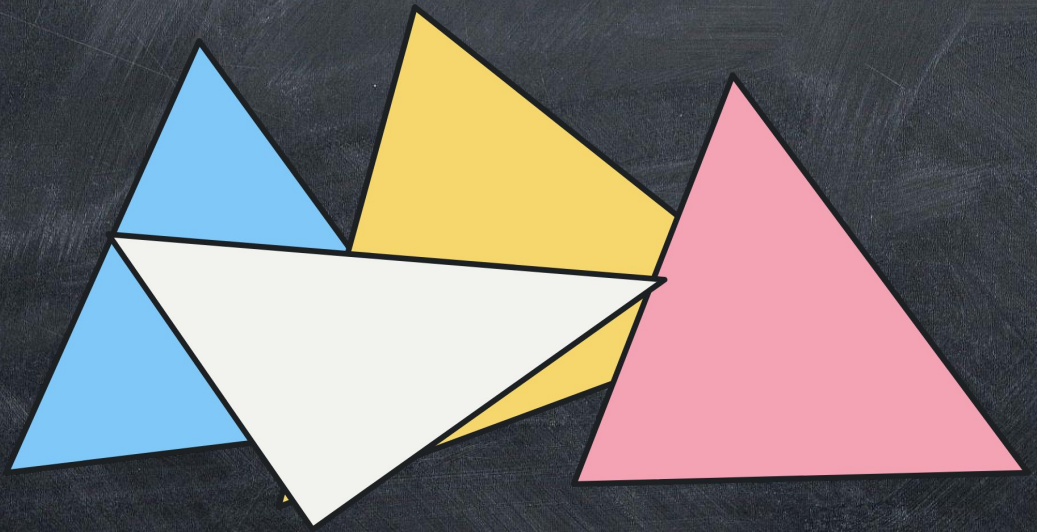
36 pixels
pixel shader ran 36 times



Full resolution
same shader, more pixels

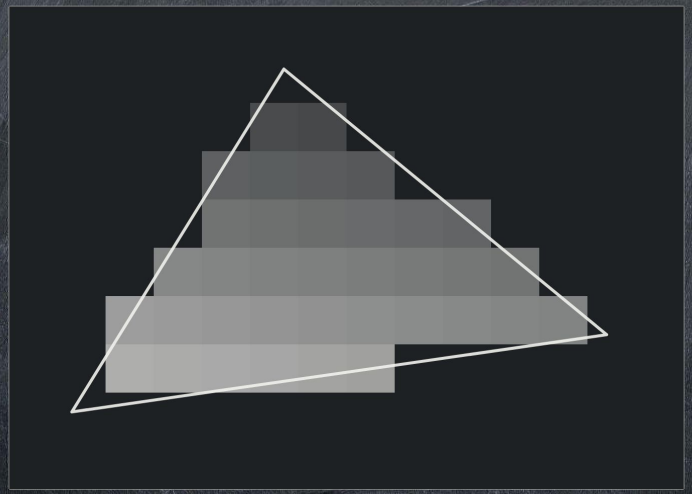
Depth

- That's one triangle. The renderer keeps drawing triangles. What happens when they overlap?



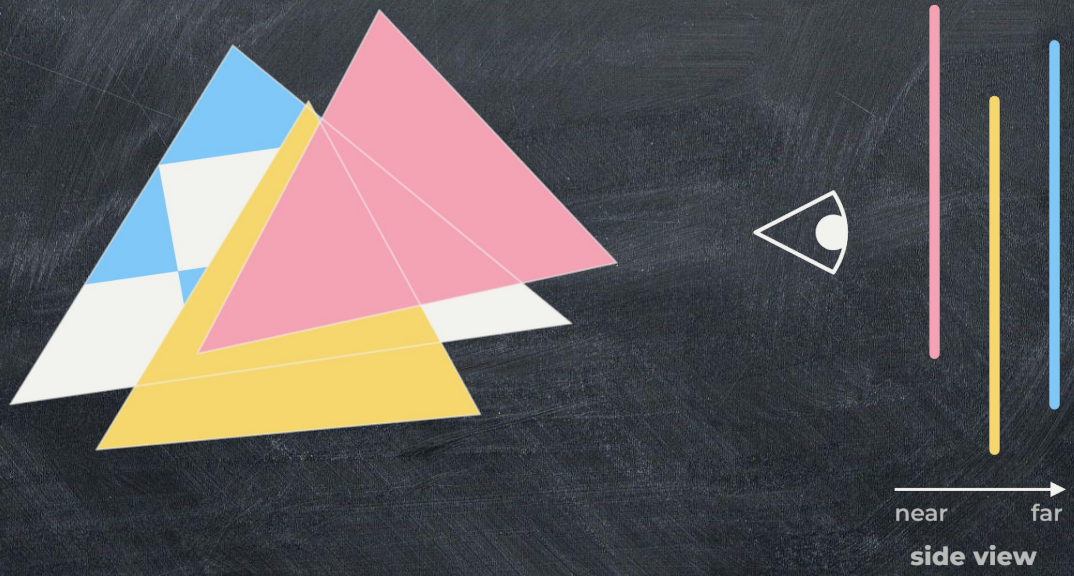
Depth

- **The framebuffer stores a depth along with each color.**



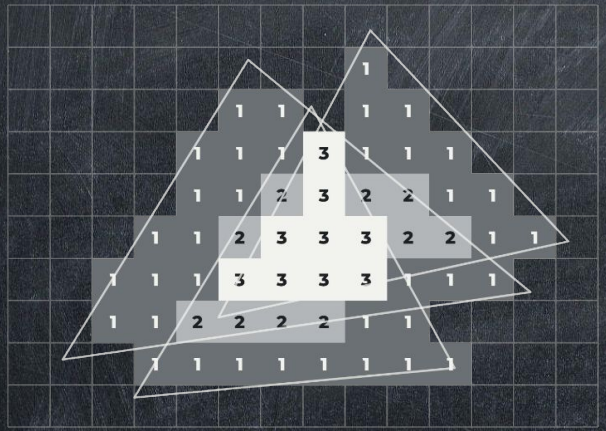
near far

- **Each pixel keeps whichever triangle is closest.**
- **Early-Z: hidden pixels skip the pixel shader.**

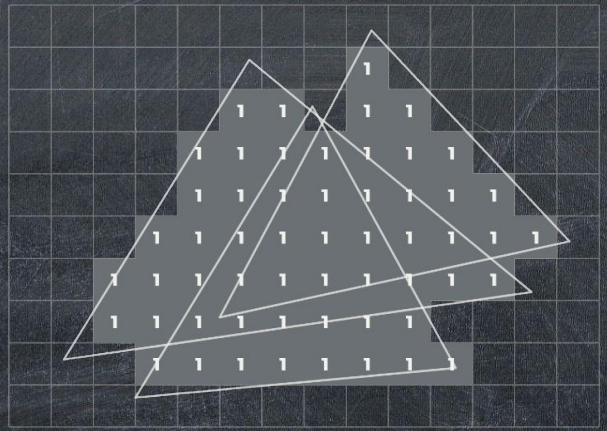


Overdraw

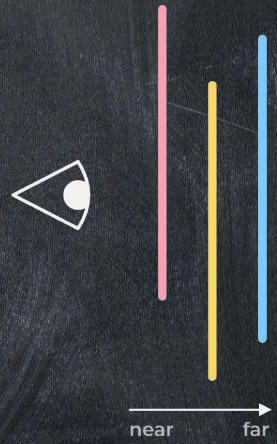
- **Draw order is important, and could shade the same pixel more than once.**



Back to front
every layer gets shaded



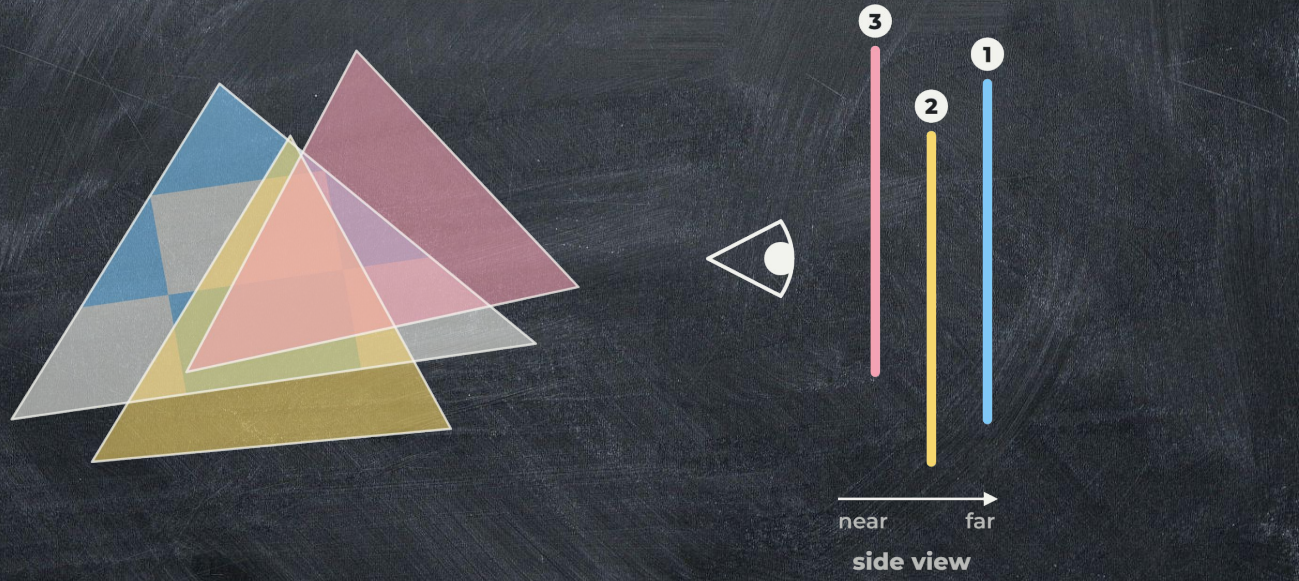
Front to back + early-Z
each pixel shaded once



side view

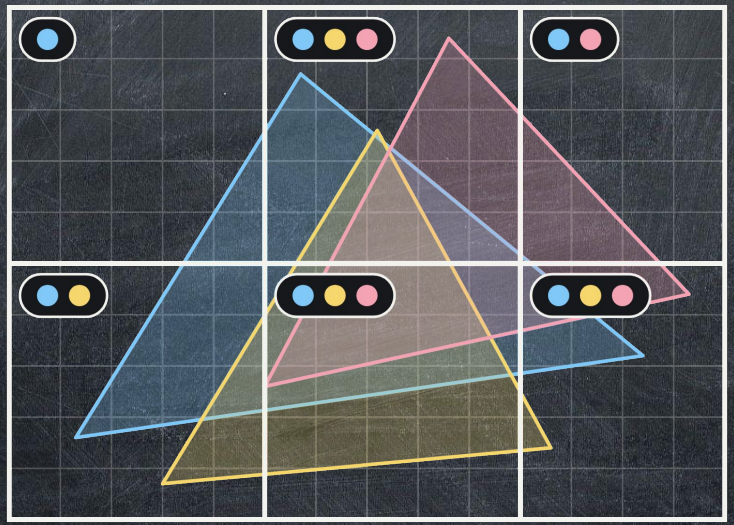
Transparency

- **Transparent objects blend with what's behind them, so draw them back to front (the Painter's Algorithm).**



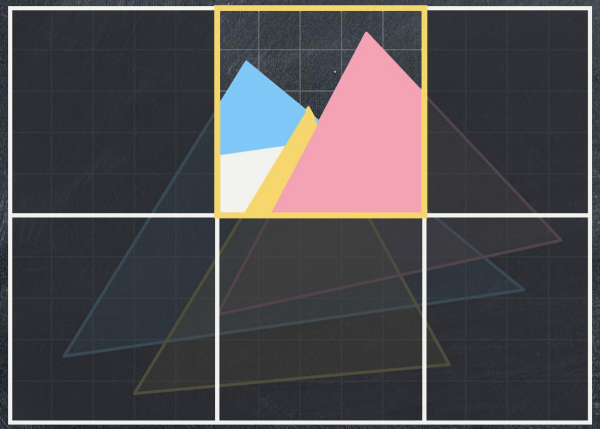
Mobile GPUs

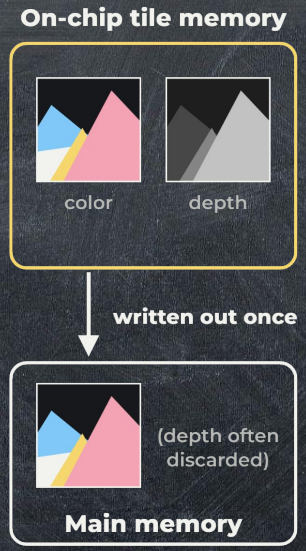
- **Memory is at a premium here. Through binning, the renderer finds all triangles in a tile on the screen.**



Mobile GPUs

- **GPU draws each tile one at a time on faster memory, then assembles them together.**





Thank you!

Diagrams inspired by LearnOpenGL.com